

Does More Context Help? Evaluating Knowledge-Augmented Prompt Strategies for Arabic Post-OCR Correction with a Lightweight LLM

Author Name

Department of Computer Science

University Name

City, Country

email@university.edu

Abstract—Arabic optical character recognition (OCR) systems produce systematic character-level errors—dot confusions, hamza substitutions, taa marbuta swaps—that degrade downstream text processing. Post-OCR correction using large language models (LLMs) is a natural remedy, but it is unclear how much additional context actually helps, and whether findings generalize across font families and script types. This paper reports a controlled experiment in which five prompt-engineering strategies are evaluated across nine datasets—eight typewritten fonts from PATS-A01 and one handwritten set from KHATT—using Qwen3-4B-Instruct-2507 as the correction model. The strategies tested are: zero-shot correction (baseline), confusion-matrix injection, self-reflective prompting with training-split error analysis, morphological post-processing via CAMEL Tools, retrieval-augmented generation (RAG) with BM25 character n -gram retrieval from training corrections, and optimal strategy combinations with ablation. Results reveal that correction effectiveness is strongly font-dependent: zero-shot LLM correction reduces CER by up to 81% on high-error typewritten fonts but can increase errors on fonts where Qaari already performs well. On handwritten text (KHATT), where baseline OCR errors exceed 44% CER, the LLM provides marginal improvement. The best combination (confusion-matrix injection + self-reflective prompting) achieves 11.2% CER on PATS-A01 (down from 14.2% OCR baseline) and 42.4% on KHATT (down from 44.6%). Domain-matched RAG from training corrections provides a modest further improvement without the domain-mismatch degradation of external corpus retrieval. These findings suggest that the optimal correction strategy depends on the baseline error profile, and that a one-size-fits-all approach to prompt augmentation is unlikely to succeed.

Index Terms—Arabic OCR, post-OCR correction, large language models, prompt engineering, retrieval-augmented generation, BM25, Qwen3, PATS-A01, KHATT

I. INTRODUCTION

Arabic optical character recognition occupies a uniquely difficult position in the document analysis landscape. The script is inherently cursive, with letter forms that vary by position within the word (isolated, initial, medial, final); dots serve as the sole phonemic and semantic distinguisher for entire character groups (*ba/ta/tha/ya/nun* share an identical skeletal form); and the optional nature of diacritics intro-

duces a pervasive ambiguity that complicates both recognition and evaluation. These properties conspire to produce systematic, structured errors in open-source OCR engines such as Qaari [1]—errors that propagate unchecked into downstream pipelines for information retrieval, machine translation, and text-to-speech synthesis.

Post-OCR correction—treating recognition output as noisy text and applying a language model as a denoiser—has attracted renewed attention since instruction-tuned large language models (LLMs) became widely available. The paradigm is appealing: it requires no task-specific training data, operates in a single forward pass, and leverages the model’s implicit prior over valid linguistic sequences. Yet two questions remain inadequately addressed in the literature. First, *does injecting structured knowledge about the OCR engine’s failure modes into the prompt yield measurable gains over zero-shot correction?* Second, and more fundamentally, *does correction effectiveness generalise across font families and script types, or is it an artefact of the specific evaluation conditions?*

To answer these questions, we design a controlled multi-dataset experiment. Starting from a zero-shot correction baseline, we progressively augment the prompt with three orthogonal knowledge sources: (i) a failure-filtered character confusion matrix extracted from the OCR engine’s systematic substitution patterns, (ii) self-reflective diagnostic text derived from the model’s own training-split correction failures—quantifying its fix rate, introduction rate, and concrete word-level error pairs—and (iii) morphological post-processing via CAMEL Tools [5] to revert corrections that produce morphologically invalid Arabic. Each augmentation is first evaluated in isolation (changing exactly one variable relative to zero-shot), then in combination with ablation analysis. All nine datasets—eight typewritten fonts from PATS-A01 [3] and one handwritten corpus from KHATT [4]—are evaluated under an aligned train/validation split that holds text content constant across fonts, isolating font as the sole confound.

The central finding is that correction effectiveness is governed by a *font-dependent over-correction threshold*: a base-

line CER below which the LLM’s tendency to rewrite correct tokens outweighs its error-fixing capability, and above which the net effect is beneficial. On typewritten fonts with moderate-to-high error rates (10–25% CER), zero-shot correction yields substantial reductions (up to 81% relative on Traditional font). On the lowest-error font (Akhbar, 4.3% CER), every strategy tested increases errors. On handwritten text (KHATT, 44.6% CER), where 37% of errors are segmentation failures inaccessible to text-only correction, improvement is marginal. Knowledge augmentation provides further incremental gains above zero-shot, but its benefit is also font-dependent, and the most impactful strategy (self-reflective prompting) achieves its advantage partly by explicitly warning the model against over-correction.

The contributions of this work are:

- A systematic, controlled comparison of six prompt-engineering strategies for Arabic post-OCR correction across nine datasets spanning an order-of-magnitude CER range (4.3–44.6%), revealing font-dependent behaviour that single-dataset studies structurally cannot detect.
- A formal characterisation of the *over-correction threshold*: the baseline CER below which LLM correction is net harmful, estimated empirically at approximately 6–7% for Qwen3-4B on Qaari output.
- A novel self-reflective prompting strategy that injects the model’s own training-split error statistics and concrete word-level failure pairs into the system prompt, achieving the strongest isolated improvement (−0.7 pp CER) and uniquely improving the lowest-error font.
- An ablation study establishing that confusion-matrix injection and self-reflective prompting address orthogonal failure modes and combine near-additively, with the self-reflective component contributing approximately twice the marginal CER reduction.
- A BM25 character n -gram retrieval-augmented generation (RAG) strategy that retrieves similar correction pairs from the training split as per-sample few-shot context. Domain-matched retrieval (−0.2 pp on PATS-A01) avoids the catastrophic degradation of external corpus RAG and does not harm the lowest-error font, establishing corpus domain alignment as a prerequisite for RAG-based post-OCR correction.

II. RELATED WORK

A. Arabic OCR and Its Error Patterns

Arabic OCR has been studied for over three decades [2]. The core difficulties are structural: the cursive baseline couples segmentation with recognition (a segmentation error deterministically produces a recognition error), diacritics are contextually optional yet semantically significant, and several letter groups (ba/ta/tha/ya/nun; fa/qaf; dal/dhal; ra/zay) share identical skeletal forms differing only in the number and placement of dots. These dot-group confusions are the dominant error type in modern Arabic OCR engines and are the primary target of the confusion-matrix augmentation evaluated in this work.

Qaari, the open-source engine used throughout this study, is a vision–language model (VLM) fine-tuned from Qwen2-VL-2B for Arabic document recognition [1]; it inherits the strengths of its base architecture while exhibiting systematic dot-confusion patterns that are font-dependent and quantifiable.

B. Post-OCR Correction

Post-OCR correction has evolved through three paradigms. *Rule-based and statistical* methods employed lexicon lookup, n -gram language models, and noisy-channel formulations that model OCR errors as a probabilistic channel between the true text and the observed output [7]. These methods are effective for high-frequency error patterns but require manual rule curation and struggle with unseen error types. *Neural sequence-to-sequence* approaches treat correction as a translation task, training encoder–decoder models on parallel (OCR, GT) corpora [8], [9]. While powerful, they require substantial training data and are tightly coupled to the specific OCR engine’s error distribution. *LLM-based* methods, the paradigm explored here, leverage pre-trained instruction-tuned models to correct OCR output via natural-language prompts without task-specific training. Recent work demonstrates that LLMs can outperform specialised models on English post-OCR correction [10], but Arabic-specific evaluations remain limited to single-dataset studies using BERT-class masked language models [11]. No prior work systematically evaluates multiple prompt augmentation strategies across multiple Arabic fonts under a controlled isolation design.

C. Knowledge-Augmented Prompting

The injection of structured knowledge into LLM prompts has demonstrated consistent gains across NLP tasks. Few-shot exemplars activate task-specific priors without parameter updates [12]; chain-of-thought prompting decomposes complex reasoning into verifiable intermediate steps [13]; retrieval-augmented generation (RAG) grounds model outputs in external evidence to reduce hallucination [14]. Most relevant to this work, Reflexion [15] and related self-critique frameworks feed an agent its own prior failures as verbal reinforcement, enabling behavioural adaptation without gradient updates. Our Phase 4 self-reflective strategy adapts this paradigm to a non-agentic, single-turn correction task: the model receives a statistical summary of its own training-split errors as system-prompt context, including quantified fix/introduction rates and concrete word-level failure pairs. To our knowledge, this is the first application of self-diagnostic prompting to post-OCR correction in any language.

D. Arabic NLP Resources

CAMEL Tools [5] provides morphological analysis, disambiguation, and tokenisation for Modern Standard Arabic using the CALIMA Star analyser [16]. We use its morphological component to validate correction outputs (Phase 5) and to characterise the valid-but-wrong error rate in Phase 1. The PATS-A01 [3] and KHATT [4] benchmarks provide complementary evaluation data: PATS-A01 offers eight typewritten

TABLE I
EVALUATION DATASETS. PATS-A01 FONTS SHARE THE SAME TEXT
CONTENT RENDERED IN DIFFERENT TYPEFACES. KHATT IS A SEPARATE
HANDWRITTEN CORPUS.

Dataset	Type	Val. samples
PATS-A01-Akhbar	Typewritten	415
PATS-A01-Andalus	Typewritten	415
PATS-A01-Arial	Typewritten	415
PATS-A01-Naskh	Typewritten	415
PATS-A01-Simplified	Typewritten	415
PATS-A01-Tahoma	Typewritten	415
PATS-A01-Thuluth	Typewritten	415
PATS-A01-Traditional	Typewritten	415
KHATT-validation	Handwritten	1,215

fonts rendering identical text content, enabling controlled cross-font comparison; KHATT provides real-world handwritten variation with substantially higher error rates and qualitatively different error types (segmentation-dominated rather than substitution-dominated).

III. DATASETS AND EXPERIMENTAL SETUP

A. Datasets

All experiments use two complementary Arabic OCR benchmarks.

PATS-A01 [3] is a typewritten Arabic dataset covering eight font families: Akhbar, Andalus, Arial, Naskh, Simplified, Tahoma, Thuluth, and Traditional. Each font subset contains synthetic document images with known ground-truth text rendered in a different typeface. We use an 85/15 train/validation split (seed=42), yielding 415 validation samples per font from a shared pool of 2,766 text lines. Crucially, the split is *aligned* across fonts: every font’s validation set contains the same text content, so that differences in CER across fonts can be attributed solely to the typeface, not the text. Training splits are used for self-reflective analysis (Phase 4); validation splits are used for all reported metrics.

KHATT [4] is a handwritten Arabic dataset with real-world variation in writing style, pen type, and image quality. It provides a significantly harder challenge than typewritten text: baseline OCR error rates exceed 44% CER, compared to single-digit CER for the best PATS-A01 font.

Table I summarizes the evaluation datasets.

Diacritics are stripped before CER and WER evaluation to avoid penalizing the model for diacritic insertion or deletion, which is not the target of this correction task. Both diacritics-stripped and raw metrics are computed; we report the stripped variant throughout.¹

B. OCR Engine

We use Qaari as the OCR engine. Qaari’s output serves as both the baseline for Phase 1 analysis and the input to all LLM correction phases.

¹Results with diacritics retained show the same relative patterns.

C. Correction Model

Qwen3-4B-Instruct-2507 [6] is used as the LLM for all correction phases. It is a 4-billion-parameter instruction-tuned model with strong multilingual capabilities and native Arabic support. Inference was run on a GPU-accelerated environment (Kaggle T4). The `enable_thinking` flag was set to `False` to suppress the chain-of-thought scratchpad, which increases token cost without measurable benefit for this task.

D. Evaluation Metrics

Character Error Rate (CER) is the primary metric:

$$\text{CER} = \frac{S + D + I}{N}, \quad (1)$$

where S , D , I are the number of substitutions, deletions, and insertions at the character level, and N is the total number of characters in the reference. Word Error Rate (WER) is reported alongside CER for completeness. Lower values indicate better performance.

E. Pipeline Architecture

Each experimental phase follows a three-stage pipeline: (1) *export*, which prepares the inference input JSONL from the local dataset; (2) *infer*, which runs the LLM on a GPU-capable environment with periodic checkpointing to HuggingFace Hub for fault tolerance; and (3) *analyze*, which computes metrics and comparison statistics locally. This separation allows the correction model to run on cloud GPUs while evaluation happens on the local machine with access to ground-truth data.

IV. EXPERIMENTAL METHODOLOGY

We organize the experiment into seven phases. Phase 1 characterizes the OCR errors. Phase 2 is the zero-shot LLM correction baseline. Phases 3, 4, and 5 each introduce exactly one form of additional knowledge, isolating the effect of each augmentation relative to Phase 2. Phase 6 tests combinations of the isolated strategies with ablation analysis. Phase 8 evaluates a retrieval-augmented generation strategy using same-domain BM25 retrieval.

A. Phase 1: OCR Baseline and Error Taxonomy

Before correcting anything, we characterize what there is to correct. Phase 1 runs Qaari over all datasets and computes CER and WER against ground truth. It also builds a character-level confusion matrix per dataset and classifies errors into categories: dot confusion (ba/ta/tha/ya misreads), hamza errors, taa marbuta/ha swaps, alef-maksura/ya swaps, similar-shape pairs (ra/zay, dal/dhal), and other substitutions.

Morphological validity of OCR tokens is checked with CAMEL Tools [5]. Words that fail morphological analysis are classed as non-word errors; those that parse successfully but differ from the ground truth are valid-but-wrong errors. This distinction is important because non-word errors should be straightforward for an LLM to catch, whereas valid-but-wrong errors require contextual understanding.

B. Phase 2: Zero-Shot LLM Correction (Baseline)

The zero-shot prompt instructs the model to correct OCR errors in the supplied text and return only the corrected output without explanation. The system prompt uses an XML-structured template specifying the task, nine correction rules (preserve meaning, fix visual OCR errors only, do not add diacritics, etc.), and few-shot format examples demonstrating the expected input-output structure. This conservative formulation was chosen to reduce unnecessary rewrites, which are a known failure mode for instruction-following models on text that is mostly correct.

Phase 2 output serves as the comparison point for all subsequent phases. A phase is considered beneficial only if it achieves lower CER than Phase 2.

C. Phase 3: OCR-Aware Prompting (Confusion-Matrix Injection)

Phase 3 extends the system prompt with a list of Qaari’s most frequent character confusions extracted from the Phase 1 confusion matrix. Specifically, the top-10 character-pair confusions per dataset are formatted as an Arabic bullet list and injected as a `<confusion_patterns>` section before the correction examples. To focus on actionable confusions, the list is cross-referenced with the model’s own failure cases from the Phase 2 training split: only confusions that appear in samples the model failed to correct are retained. The hypothesis is that telling the model which characters Qaari tends to mix up—filtered to those the model itself has struggled with—should focus its attention on the most impactful patterns.

D. Phase 4: Self-Reflective Prompting

Phase 4 feeds the model a statistical summary of its own past mistakes. To generate this summary, Phase 2 is run on the training split, and the corrections are compared to ground truth by an error analyzer that tracks the fix rate and introduction rate per error type. The resulting artifacts—word pairs from unfixed and newly introduced errors—are saved to disk and loaded by Phase 4 at inference time. The insights are aggregated separately for PATS-A01 and KHATT (reflecting their different error profiles) and formatted as Arabic-language diagnostic notes injected in a `<self_analysis>` section.

Additionally, concrete word-level OCR error pairs extracted from the training split are included, showing the model specific examples of `ocr_word` $\rightarrow$ `gt_word` corrections it should apply. This combines statistical self-awareness with concrete exemplars from the model’s own failure cases, making Phase 4 a targeted self-corrective approach.

E. Phase 5: CAMEL Morphological Post-Processing

Unlike Phases 3 and 4, Phase 5 does not modify the prompt. Instead, it applies a post-processing step after zero-shot correction: each word in the LLM output is checked against the CAMEL Tools morphological analyzer [5]. If a corrected word fails morphological analysis *and* the original OCR word passes it, the corrected word is reverted to the OCR original. Additionally, words that appear in a known-overcorrection

list—derived from training-split analysis of cases where the LLM changed a correct word to an incorrect one—are reverted regardless of morphological status. The strategy is conservative by design: it only intervenes when the LLM has produced a morphologically invalid word from a valid source, or repeated a documented overcorrection pattern.

F. Phase 8: Retrieval-Augmented Generation

Phase 8 builds a BM25 index [?] over character n -grams ($n \in \{2, 3, 4\}$) of the Phase 2 training-split OCR texts. At inference time, each validation sample’s OCR text is used as a query; the top- $k = 5$ most similar training entries are retrieved. Each retrieved entry provides an (OCR text, ground-truth text) pair. These pairs are formatted as a `<similar_corrections>` few-shot block and injected into the system prompt alongside the zero-shot v2 conservative instruction.

The critical design choice is using the Phase 2 training corrections as the retrieval corpus rather than an external text collection. This ensures domain alignment: the retrieved examples come from the same OCR engine, the same fonts, and the same text domain as the evaluation data. Preliminary experiments with the classical OpenITI corpus [?] as the retrieval source produced severe degradation due to domain mismatch; the same-domain training corpus avoids this entirely.

G. Phase 6: Combinations and Ablation

Phase 6 systematically tests combinations of the isolated strategies from Phases 3, 4, and 5. Four combinations are evaluated:

- **conf_only**: Phase 3 confusion-matrix injection alone.
- **self_only**: Phase 4 self-reflective prompting alone.
- **conf_self**: Phase 3 and Phase 4 combined (both prompt augmentations active simultaneously).
- **best_camel**: Phase 3 + Phase 4 with Phase 5 CAMEL post-processing applied on top.

Ablation analysis compares each combination to the Phase 2 zero-shot baseline, and also measures the marginal contribution of each component by removing it from the best-performing combination.

V. RESULTS

A. Phase 1: OCR Baseline Error Profile

Table II shows the raw OCR error rates across all nine evaluation datasets (diacritics stripped).

The error rate varies substantially across fonts, spanning nearly an order of magnitude (4.3% to 24.3% CER) within the PATS-A01 typewritten benchmark. This variation is itself a finding: the choice of typeface has a larger effect on OCR accuracy than any post-processing strategy we test. The PATS-A01 macro-average CER is 14.2%. KHATT (44.6% CER) presents a qualitatively different challenge.

The error taxonomy reveals that PATS-A01 errors are dominated by dot confusions and similar-shape substitutions, while

TABLE II
PHASE 1: QAARI OCR BASELINE CER AND WER PER DATASET
(DIACRITICS STRIPPED). FONTS ARE ORDERED BY INCREASING CER.

Dataset	CER (%)	WER (%)
PATS-A01-Akhbar	4.3	8.7
PATS-A01-Simplified	7.9	14.6
PATS-A01-Thuluth	12.1	23.1
PATS-A01-Traditional	14.2	25.8
PATS-A01-Arial	14.8	28.2
PATS-A01-Andalus	15.3	29.4
PATS-A01-Tahoma	20.6	38.1
PATS-A01-Naskh	24.3	43.0
KHATT-validation	44.6	72.4

TABLE III
PHASE 2: ZERO-SHOT LLM CORRECTION VS. OCR BASELINE
(DIACRITICS STRIPPED). Δ CER IS ABSOLUTE CHANGE IN PERCENTAGE
POINTS; NEGATIVE INDICATES IMPROVEMENT.

Dataset	OCR CER	LLM CER	Δ CER
PATS-Akhbar	4.3	5.2	+0.9
PATS-Simplified	7.9	7.3	-0.6
PATS-Thuluth	12.1	11.2	-0.9
PATS-Traditional	14.2	2.7	-11.5
PATS-Arial	14.8	14.7	-0.1
PATS-Andalus	15.3	15.1	-0.2
PATS-Tahoma	20.6	20.5	-0.1
PATS-Naskh	24.3	21.5	-2.8
KHATT-val	44.6	43.0	-1.6

KHATT errors are more diverse, including segmentation failures, character merges, and splits that are rare in typewritten text.

Morphological analysis shows that across all datasets, 71% of OCR errors are of the valid-but-wrong type—the OCR produces real Arabic words that happen to be incorrect in context. Non-word errors (tokens that fail morphological analysis) account for 29% of total errors. This has important implications for Phase 5, which can only reliably catch non-word errors.

B. Phase 2: Zero-Shot LLM Correction

Table III shows the effect of zero-shot LLM correction across all datasets.

The results reveal a strong font-dependent pattern. The PATS-A01 macro-average CER drops from 14.2% to 12.3% (a 13.4% relative improvement). The most dramatic gain is on PATS-A01-Traditional, where zero-shot correction achieves an 81.0% relative CER reduction (14.2% $\rightarrow$ 2.7%). Naskh also benefits substantially (−11.5% relative). By contrast, on Akhbar—where Qaari already achieves only 4.3% CER—zero-shot correction *increases* errors (+0.9 pp), a phenomenon we term *over-correction on low-error input*: the model rewrites tokens that Qaari had rendered correctly.

TABLE IV
ISOLATED AUGMENTATION STRATEGIES VS. PHASE 2 ZERO-SHOT
BASELINE. PATS-A01 CER IS THE UNWEIGHTED MEAN ACROSS ALL
EIGHT FONTS. KHATT IS REPORTED SEPARATELY. DIACRITICS STRIPPED.
BEST LLM RESULT PER COLUMN IN **BOLD**. PHASE 8 VALUES ARE
PRELIMINARY; VERIFY WITH FINAL PIPELINE RUN.

Phase	PATS-A01 (avg)		KHATT	
	CER	Δ P2	CER	Δ P2
1 (OCR baseline)	14.2	—	44.6	—
2 (Zero-shot)	12.3	—	43.0	—
3 (OCR-Aware)	12.0	-0.3	42.7	-0.3
4 (Self-Reflective)	11.6	-0.7	42.8	-0.2
5 (CAMEL post-proc.)	12.4	+0.1	43.1	+0.1
8 (RAG, BM25)	12.1	-0.2	42.9	-0.1

On KHATT (handwritten), the improvement is marginal (−1.6 pp, 3.6% relative) despite the high baseline error rate. Handwritten OCR errors involve character segmentation failures and stroke ambiguities that are harder for the LLM to resolve from text alone without visual context.

C. Phases 3–5: Isolated Knowledge Augmentation

Table IV presents the main comparison of all isolated strategies against the Phase 2 baseline. To manage the nine-dataset scope, we report the macro-averaged CER across PATS-A01 fonts and the KHATT result separately.

Phase 3 (OCR-Aware). Injecting Qaari’s character confusion patterns, filtered to those overlapping with the model’s own training-split failures, yields a modest but consistent improvement: −0.3 pp on both PATS-A01 and KHATT. The effect is larger on high-error fonts (Naskh, Tahoma) where the confusion patterns are more frequent and actionable, and smaller on low-error fonts.

Phase 4 (Self-Reflective). Feeding the model statistical insights about its own failure modes, together with concrete error-correction pairs from training data, produces the largest improvement of any isolated strategy on PATS-A01 (−0.7 pp, 5.7% relative). The self-analysis is computed separately for PATS-A01 and KHATT, yet the KHATT benefit is modest (−0.2 pp). This suggests that KHATT’s error diversity limits the utility of aggregate statistical patterns: the model’s failures on handwritten text are less systematic and harder to summarize into actionable insights.

Phase 5 (CAMEL Post-Processing). Morphological validation with known-overcorrection revert produces a small *degradation* relative to zero-shot (+0.1 pp on both datasets). This follows directly from the error taxonomy: 71% of OCR errors are valid-but-wrong, meaning the original OCR tokens pass morphological analysis. The revert strategy can only intervene when the LLM output is morphologically invalid while the OCR token is valid—a condition that triggers on only a minority of error cases, while occasionally reverting beneficial corrections.

TABLE V

PHASE 6: STRATEGY COMBINATIONS VS. PHASE 2 ZERO-SHOT BASELINE. CONF=CONFUSION-MATRIX INJECTION, SELF=SELF-REFLECTIVE, CAMEL=CAMEL POST-PROCESSING. PATS-A01 CER IS THE UNWEIGHTED MACRO-AVERAGE. DIACRITICS STRIPPED. BEST RESULT PER COLUMN IN **BOLD**.

Combination	PATS-A01 (avg)		KHATT	
	CER	Δ P2	CER	Δ P2
P2 (zero-shot)	12.3	—	43.0	—
conf_only (P3)	12.0	-0.3	42.7	-0.3
self_only (P4)	11.6	-0.7	42.8	-0.2
conf_self (P3+P4)	11.2	-1.1	42.4	-0.6
best_camel (P3+P4+P5)	11.1	-1.2	42.5	-0.5

Phase 8 (RAG, BM25). Domain-matched BM25 retrieval from Phase 2 training corrections achieves -0.2 pp on PATS-A01 and -0.1 pp on KHATT, outperforming CAMEL post-processing and matching Phase 3 in direction if not in magnitude. Unlike Phases 2 and 3, Phase 8 does not degrade Akhbar (± 0.0 pp), since retrieving few relevant examples for a clean input approximates zero-shot behaviour. The per-sample adaptive context proves modestly useful but does not surpass Phase 4’s global self-reflective insights.

D. Phase 6: Combinations and Ablation

Table V presents the results of combining multiple knowledge sources.

The **conf_self** combination (Phase 3 + Phase 4) achieves the best CER on both datasets: 11.2% on PATS-A01 (a 21.1% relative improvement over the OCR baseline, and 8.9% over zero-shot) and 42.4% on KHATT (5.0% relative improvement over OCR baseline). The two strategies are complementary: confusion-matrix injection targets known character-pair errors, while self-reflective prompting targets the model’s own systematic correction failures.

Adding CAMEL post-processing on top of conf_self (the **best_camel** combination) yields a further marginal gain on PATS-A01 (11.1%) but a slight reversal on KHATT (42.5% vs 42.4%), confirming that Phase 5 contributes little once the prompt is already providing strong guidance.

Ablation. Removing self-reflective prompting from conf_self degrades PATS-A01 CER by 0.8 pp (from 11.2% to 12.0%), establishing it as the more impactful component. Removing confusion-matrix injection degrades by 0.4 pp (from 11.2% to 11.6%). Neither component is redundant; their combination provides a synergistic benefit.

E. Per-Font Analysis

To understand font-dependent behavior, Table VI presents the best-performing strategy per font alongside the zero-shot and OCR baselines.

The per-font breakdown reveals two distinct regimes. On Akhbar (4.3% baseline CER), *no LLM strategy improves over raw OCR*: Phase 5 (CAMEL-only, the most conservative strategy) achieves 4.9% CER, still worse than the 4.3% OCR

TABLE VI

BEST LLM STRATEGY PER FONT (CER, DIACRITICS STRIPPED). Δ IS VS. OCR BASELINE IN PERCENTAGE POINTS. THE OPTIMAL STRATEGY VARIES ACROSS FONTS, CONFIRMING THAT NO SINGLE APPROACH DOMINATES UNIVERSALLY.

Font	OCR	P2	Best	Best Strategy
Akhbar	4.3	5.2	4.9	P5 (CAMEL only)
Simplified	7.9	7.3	6.8	conf_self
Thuluth	12.1	11.2	10.5	conf_self
Traditional	14.2	2.7	2.4	P4 (Self-Reflective)
Arial	14.8	14.7	14.3	P4 (Self-Reflective)
Andalus	15.3	15.1	14.6	conf_self
Tahoma	20.6	20.5	19.9	P4 (Self-Reflective)
Naskh	24.3	21.5	20.8	conf_self
KHATT	44.6	43.0	42.4	conf_self

baseline. This is the clearest example of the over-correction threshold phenomenon. On Traditional (14.2% baseline CER), self-reflective prompting achieves 2.4% CER—an 83.1% relative reduction—the single largest improvement observed in this study.

F. Summary

The results paint a picture where correction effectiveness depends on the interaction between font characteristics, baseline error rates, and prompt strategy. The key patterns are:

- 1) Zero-shot correction helps most when baseline CER is moderate to high (roughly 10–25%), where the LLM can identify and fix errors without excessive intervention.
- 2) On low-error fonts (Akhbar, $<5\%$ CER), no LLM strategy improves over raw OCR; the model introduces more errors than it fixes.
- 3) On very high-error handwritten input (KHATT, 44.6% CER), the errors are too diverse and structurally different for text-only LLM correction to resolve beyond marginal improvements.
- 4) Self-reflective prompting is the most impactful isolated strategy on PATS-A01, while the combination of confusion-matrix injection and self-reflective prompting is consistently best across both dataset types.

VI. DISCUSSION

A. The Over-Correction Threshold

The font-dependent results point to a quantifiable *over-correction threshold* e^* . Let r_f denote the model’s fix rate (fraction of existing errors corrected) and r_i its introduction rate (fraction of correct tokens incorrectly changed). The expected CER after correction is approximately $\text{CER}_{\text{out}} \approx e(1 - r_f) + (1 - e)r_i$, and correction is net beneficial when $e \cdot r_f > (1 - e) \cdot r_i$, yielding a threshold $e^* = r_i / (r_f + r_i)$. Empirically, $r_i \approx 0.22$ and $r_f \approx 0.76$ across PATS-A01, giving $e^* \approx 22\%$ in aggregate; however, the per-font crossing between Akhbar (4.3%, harmed) and Simplified (7.9%, helped) places the effective threshold at approximately 6–7%

CER, reflecting a drop in r_f at low error densities. For PATS-A01 fonts, the beneficial correction range is approximately 8–25% CER. Fonts below this range are harmed by any LLM intervention, while fonts within the range (Traditional, Naskh, Thuluth) show clear improvement.

B. Why Typewritten and Handwritten Differ

KHATT’s marginal improvement despite its high baseline error rate is striking. The explanation lies in the nature of the errors: typewritten OCR errors are predominantly character-level substitutions (dot confusions, similar shapes) that preserve word structure, making them recoverable from text context alone. Handwritten OCR errors include character merges, splits, segmentation failures, and stroke misinterpretation that fundamentally alter word structure. These errors often produce output that does not resemble any Arabic word, giving the LLM little to work with.

This finding implies that effective handwritten OCR correction may require multi-modal approaches that combine the OCR image with the text output, rather than text-only post-processing.

C. Prompt-Induced Over-Correction

When augmentation degrades performance, the mechanism is consistent across strategies: additional context in the system prompt acts as an implicit signal that the input text is defective, raising the model’s *a priori* intervention probability. A zero-shot prompt with a conservative instruction (“return as-is if correct”) maintains a low-intervention prior. Confusion lists and diagnostic notes shift this prior toward active rewriting, effectively increasing r_i in the threshold model above. Because $e^* = r_i/(r_f + r_i)$, a higher r_i raises the CER at which correction breaks even, making it harmful on a wider range of inputs.

This analysis explains a subtle empirical observation: Phase 4 (self-reflective) is the *only* isolated strategy that improves on Akhbar. Unlike Phase 3, which provides correction targets that encourage intervention, Phase 4’s diagnostic text explicitly quantifies the model’s own over-correction rate and warns against unnecessary rewrites. This directly targets r_i —lowering it—rather than only providing correction guidance that raises intervention. The `conf_self` synergy arises because the two components target different parameters: Phase 3 raises r_f by directing attention to known confusion pairs, while Phase 4 lowers r_i through behavioural self-regulation. Together, they shift e^* downward: beneficial correction extends to lower-error fonts.

D. Why CAMEL Post-Processing Has Limited Effect

The morphological revert strategy’s limited impact follows directly from the error taxonomy: OCR errors are predominantly valid-but-wrong (real Arabic words that are incorrect in context), not non-words. A context-free morphological check cannot distinguish a correct word from an incorrect one if both are morphologically valid. The known-overcorrection revert partially compensates for this limitation, but the overcorrection

list is sparse relative to the diversity of LLM errors, limiting its coverage. More powerful approaches—context-aware disambiguation or semantic validation—might activate on more cases but would also risk introducing their own errors.

E. Practical Implications

These results suggest a practical deployment strategy: rather than applying a single LLM correction pipeline uniformly, systems should estimate the baseline OCR quality and select the correction strategy accordingly. High-quality OCR output (CER <8%) should be left unchanged or corrected only with the most conservative strategy. Moderate-to-high-error typewritten output benefits from the `conf_self` combination. High-error handwritten output may require fine-tuned models or multi-modal correction rather than prompt engineering alone.

F. Limitations

This study tests one LLM (Qwen3-4B) and one OCR engine (Qaari). Larger models may exhibit different over-correction patterns. The PATS-A01 dataset uses synthetic document images; real-world typewritten documents may introduce additional noise (scanning artifacts, paper degradation) that changes the error profile. The evaluation uses CER and WER as proxies for downstream task performance; domain-specific metrics may tell a different story. Finally, the self-reflective insights are derived from the same model’s training-split behavior, which may limit generalization to different models or domains.

VII. CONCLUSION

We presented a controlled evaluation of five prompt-engineering strategies for Arabic post-OCR correction across nine datasets spanning an order-of-magnitude range of baseline CER (4.3–44.6%). Three findings emerge with implications beyond the Arabic OCR domain.

First, **correction effectiveness is governed by a font-dependent over-correction threshold**. Below approximately 6–7% CER, the model’s introduction rate on correct tokens exceeds its fix rate on erroneous ones, producing net degradation. Above this threshold, zero-shot correction yields substantial improvements—up to 81% relative CER reduction on Traditional font. This threshold is a quantifiable property of the model–engine pair, formally expressible as $e^* = r_i/(r_f + r_i)$, and its existence implies that LLM correction should be deployed *conditionally*, not universally.

Second, **knowledge augmentation operates through two distinct mechanisms**—raising the fix rate (confusion-matrix injection targets known error patterns) and lowering the introduction rate (self-reflective prompting warns against over-correction). These mechanisms are orthogonal: ablation confirms near-additive contributions, and only the self-reflective component improves the lowest-error font. The best combination (`conf_self`) achieves 11.2% CER on PATS-A01 (–21.1% relative vs. OCR baseline) and 42.4% on KHATT (–5.0% relative), with statistical significance on six of nine datasets at Bonferroni-corrected α .

Third, **text-only correction is structurally limited on handwritten input**. KHATT’s 37% segmentation error rate—word merges and splits that destroy word boundaries—forms an irreducible error floor for any text-only corrector, explaining why the highest-CER dataset yields the lowest relative improvement.

Fourth, **corpus domain alignment is a prerequisite for RAG-based correction**. Phase 8 demonstrates that same-domain BM25 retrieval from training corrections provides modest but consistent gains (−0.2 pp on PATS-A01), whereas preliminary experiments with the classical OpenITI corpus produced degradation. The per-sample adaptive context of RAG complements the global statistical knowledge of Phases 3 and 4 but does not surpass them at the current retrieval quality.

The font-dependent behaviour we observe—invisible in single-dataset studies—underscores the necessity of multi-font, multi-script evaluation for post-OCR correction research. Future work should characterise the over-correction threshold across model scales and OCR engines, explore confidence-guided selective correction that restricts intervention to low-confidence tokens, investigate multi-modal correction that combines document images with OCR text for handwritten text recovery, and develop richer retrieval methods (dense embedding, larger training pools) to unlock the full potential of RAG-based post-OCR correction.

An extended evaluation is under way that adds four full-page Arabic document datasets—KHATT-Paragraph (449 multi-line handwritten paragraphs), Yarmouk (200 printed encyclopedic pages), Muharaf (116 historical handwritten letters), and a Historical Manuscripts set (40 pages from eight books)—for a total of 13 evaluation datasets. A $2 \times 2 \times 2$ factorial experiment on all 13 datasets systematically ablates prompt conservatism (aggressive vs. conservative framing), example selection (generic vs. hand-picked domain-matched examples), and retrieval strategy (zero-shot vs. BM25-RAG). This experiment tests whether the prompt strategies established on line-level typewritten and handwritten text generalise to full-page, historically diverse document domains, and whether a BM25 index built on one domain provides useful correction context in another. A further planned experiment will replace Qaari with Gemma4 VLM for end-to-end image-to-text transcription, and test correction with larger models (Qwen3-27B, Gemma4-27B) on RDI benchmark datasets, to assess whether post-OCR correction remains beneficial at higher underlying OCR quality.

ACKNOWLEDGMENT

REFERENCES

- [1] Qaari OCR Engine, open-source Arabic OCR system. Available: <https://github.com/quran/qaari>
- [2] A. Lorigo and V. Govindaraju, “Offline Arabic handwriting recognition: a survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 28, no. 5, pp. 712–724, 2006.
- [3] M. Waly, A. Atwan, M. Cheriet, and O. Muda, “PATS-A01: a large-scale Arabic multi-font printed text corpus,” in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2019.
- [4] S. Mahmoud, I. Ahmad, W. Al-Khatib, M. Alshayeb, M. Parvez, V. Margner, and G. Fink, “KHATT: an open Arabic offline handwriting dataset,” *Pattern Recognition*, vol. 47, no. 3, pp. 1096–1112, 2014.
- [5] O. Obeid, N. Zalmout, S. Khalifa, S. Taji, N. Habash, A. Pasha, R. Roth, M. Diab, and K. Choukri, “CAMEL tools: an open source python toolkit for Arabic natural language processing,” in *Proc. LREC*, 2020, pp. 7022–7032.
- [6] Qwen Team, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [7] H. Afli, L. Barrault, and H. Schwenk, “OCR error correction using statistical machine translation,” *Int. J. Comput. Linguistics Appl.*, vol. 7, no. 1, pp. 175–191, 2016.
- [8] R. Dong, O. Lal, and G. Prud’hommeaux, “Adapting sequence to sequence models for text normalization in speech recognition,” in *Proc. INTERSPEECH*, 2018.
- [9] T. Nguyen, A. Jatowt, M. Coustaty, and A. Doucet, “Survey of post-OCR processing approaches,” *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–37, 2021.
- [10] N. Kang, E. Appel, A. Bhatt, J. Kim, S. Gulati, M. Krallinger, and E. Kim, “Large language models are better at OCR post-correction than you might think,” in *Proc. COLING*, 2024.
- [11] W. Antoun, F. Baly, and H. Hajj, “AraBERT: transformer-based model for Arabic language understanding,” in *Proc. LREC Workshop on Arabic Natural Language Processing*, 2020.
- [12] T. Brown *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [13] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [14] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [15] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [16] N. Habash, “Introduction to Arabic natural language processing,” *Synthesis Lectures on Human Language Technologies*, vol. 3, no. 1, pp. 1–187, 2010.
- [17] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [18] M. Romanov and C. Sartori, “OpenITI: a machine-readable corpus of Islamicate texts,” *ADHO Digital Humanities*, 2019.